	AIR UNIVERSITY
	Department Of Computer Engineering
	Experiment No : 1

Lab Tittle : Pointers, Function and Classes in C++

Name: Muhammad Waqas **Reg No:** 240756

Objectives: Use of classes and Functions in C++ using OOP.
Use of Pointers to create objects for dynamic and static memory allocation.

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks : Obtained Marks :

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: Obtained Marks :

Date: 15th September , 25 Signature : Maryam



Department of Computer Engineering

Data Structures And Algorithms

LAB REPORT : 1

Tittle : Pointers, Function and Classes in C++

Name : Muhammad Waqas

Roll No: 240756

Submitted To : CE . Engr Maryam

Date : 15h September 25

Experiment No : 1
Pointers, Functions, and Classes in C++

Task 1: Class and Object (dot operator)
Problem:
Create a class Book with members title and pages, and a function showInfo(). In main(), create an object of Book, assign values, and call the function using the **dot (.) operator**.
Expected outcome:

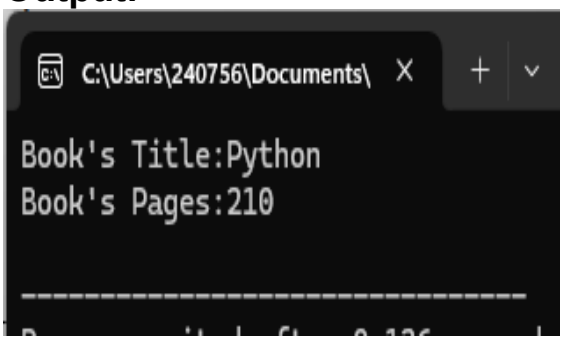
- Define a simple class with data + function
- Create object on stack
- Access with . operator

Solution:

Code:

```
#include<iostream>
using namespace std;
class Book{
private:
string Title;
int Pages;
public:
Book(string,int);
void showinfo();
};
//Parameterized Constructor
Book::Book(string t,int p){
    Title=t;
    Pages=p;}
void Book::showinfo(){
    cout<<"Book's Title:"<<Title<<endl;
    cout<<"Book's Pages:"<<Pages<<endl;}
int main(){
    Book b("Python",210);
    b.showinfo();
    return 0;}
```

Output:



Task 2: Dynamic Memory Allocation (arrow operator)
Problem:
Create a class Account with a function displayBalance().
In main(), create a pointer and use new to dynamically allocate an Account object.
Call the function using the **arrow (->) operator**.
Expected outcome:

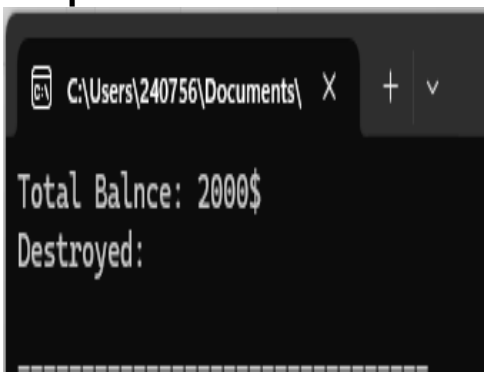
- Use new for dynamic allocation
- Call function via pointer using ->
- Free memory with delete

Solution:

Code:

```
#include<iostream>
using namespace std;
class Account{
private:
float Balance;
public:
//Parameterized Construtor
//Declaration;
Account(float);
//Destructor
~Account();
//Function to take balnce from user
void Enter_Balance();
void Display_Balance();
};
Account::~Account(){
    cout<<"Destroyed:"<<endl;
}
//Constructor Defination
Account::Account(float b1){
    Balance=b1;
}
void Account::Display_Balance(){
    cout<<"Total Balnce: "<<Balance<<"$"<<endl;;
}
int main(){
    Account* a=new  Account(2000);
    a->Display_Balance();
    //Destructor is called when object is destroyed
    delete a;
}
```

Output:



Task 3: Set and Get Functions (Encapsulation with this)
Problem:
Create a class Rectangle with private members length and width.
Provide setDimensions(int, int) and getArea() functions.
Use the this pointer inside setDimensions to assign values.
In main(), create a Rectangle object, set dimensions, and display area.
Expected outcome:

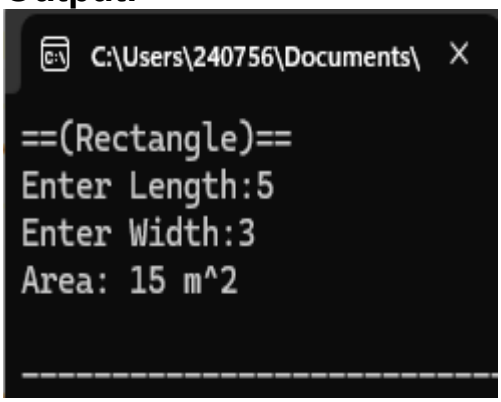
- Encapsulation with private data
- Use of set/get functions
- Apply this-> inside setter

Solution:

Code:

```
#include<iostream>
using namespace std;
class Rectangle{
private:
float Length;
float Width;
public:
void setDimension(float,float);
float get_Area();};
void Rectangle::setDimension(float l,float w){
    this->Length=l;
    this->Width=w;
}
float Rectangle::get_Area(){
    return Length*Width;
}
main(){
Rectangle r1;
float q1,q2;
cout<<"==(Rectangle)==\n";
cout<<"Enter Length:";cin>>q1;
cout<<"Enter Width:";cin>>q2;
r1.setDimension(q1,q2);
cout<<"Area: "<<r1.get_Area()<<" m^2\n";
return 0;
}
```

Output:



Task 4: Object of One Class Inside Another (Composition)
Problem:
Create a class CPU with a function showSpeed().
Create another class Computer that **has a CPU object**.
In Computer::specs(), also call cpu.showSpeed().
Expected outcome:

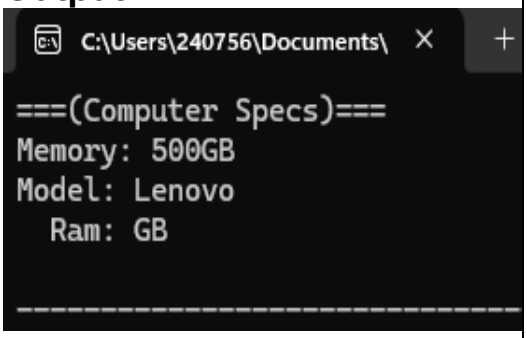
- One class contains another (composition)
- Member initializer list to initialize CPU inside Computer
- Demonstrate HAS-A relationship

- ***Solution:***

Code:

```
#include<iostream>
using namespace std;
class CPU{
private:
string model;
float memory;
public:
//Parameterized Constructor
CPU(string ,float);
void showspeed();   };
CPU::CPU(string m,float o){
this->model=m;
this->memory=o;}
void CPU::showspeed(){
cout<<"Memory: "<<memory<<"GB"<<endl;
cout<<"Model: "<<model<<endl;}
class Computer{
private:
int Ram;
public:
CPU c1;
//Inializer List
Computer(int ram,string model,float memory):
Ram(ram),c1(model,memory){}
void specs();   };
void Computer::specs(){
cout<<"===(Computer Specs)===\n";
c1.showspeed();
cout<<"  Ram: "<<"GB"<<endl;}
main(){
Computer a1(8,"Lenovo",500);
a1.specs();
}
```

Output:



Task 5 : (Home Task)
Device Management System
Problem:
Create a program that models devices and their components. It should demonstrate:

- Class and Object using the dot (.) operator.
- Dynamic Memory Allocation using new and ->.
- Set and Get Functions with encapsulation and the this pointer.
- Composition (one class containing another).
- Null Pointer Safety (checking before use).

- ***Solution:***

```
#include<iostream>
using namespace std;
class hardware {
private:
string Body_Material;
float battery_capacity;
string Connectivity;
string storage_Derive;
public:
hardware(string h1, float h2, string h3, string h4) {
this->Body_Material = h1;
this->battery_capacity = h2;
this->Connectivity = h3;
this->storage_Derive = h4;}
// Getters
string getBodyMaterial(){ return Body_Material; }
float getBatteryCapacity(){ return battery_capacity; }
string getConnectivity(){ return Connectivity; }
string getStorageDerive(){ return storage_Derive; }};
class software {
private:
string operating_System;
string Productivity_Software;
string Graphic_Software;
string Utility_Software;
public:
software(string s1, string s2, string s3, string s4) {
this->operating_System=s1;
this->Productivity_Software=s2;
this->Graphic_Software=s3;
this->Utility_Software=s4;}
//Getters
string getOperatingSystem(){return operating_System; }
string getProductivitySoftware(){return Productivity_Software; }
```

```
string getProductivitySoftware(){return Productivity_Software; }
string getGraphicSoftware(){ return Graphic_Software; }
string getUtilitySoftware(){ return Utility_Software; }};
class Devices {
private:
string Device_Name;
string Device_Function;
public:
hardware h1;
software s1;
// Initializer List
Devices(string d1, string d2,
string w1, float w2, string w3, string w4,
string w5, string w6, string w7, string w8)
:h1(w1, w2, w3, w4), s1(w5, w6, w7, w8) {
Device_Name = d1;
Device_Function = d2;}
~Devices(){}
void Device_Display();};
//Function Definition
void Devices::Device_Display() {
cout<<"Device Name: "<<Device_Name<<endl;
cout<<"Device Function: " <<Device_Function<<endl;
cout<<"\n==== (" << Device_Name << "'s Hardware)====\n";
cout<<"Body Material: "<< h1.getBodyMaterial()<<endl;
cout<<"Battery Capacity: "<< h1.getBatteryCapacity()<<endl;
cout<<"Connectivity: " <<h1.getConnectivity()<<endl;
cout<<"Storage Derive: " <<h1.getStorageDerive()<<endl;
cout<<"\n====("<< Device_Name<<"'s Software)====\n";
cout<<"Operating System: " <<s1.getOperatingSystem()<<endl;
cout<<"Productivity Software: "<<s1.getProductivitySoftware()<<endl
cout<<"Graphic Software: " <<s1.getGraphicSoftware()<<endl;
cout<<"Utility Software: "<<s1.getUtilitySoftware()<<endl;}
```

```
int main() {
Devices d("Samsung Galaxy", "Multimedia",
"Aluminium", 4500, "Bluetooth", "HDD",
"Android", "MS Office", "Adobe Photoshop", "CCleaner");
// Display device details
d.Device_Display();
cout << "\n";
// Dclare pointer and set it to nullptr
Devices* ptr = nullptr;
//Check if pointer is null
if (ptr == nullptr) {
cout << "Pointer is empty\n"<<endl;}
//Create a stack object
Devices d1("iPhone 17","Communication",
"Titanium", 5000, "Wireless","SSD",
"iOS", "Google Docs", "Nvidia Studio", "Disk Cleanup");
//Assign pointer to object
ptr = &d1;
//Call method through pointer
ptr->Device_Display();
return 0;}
```

Output:

```
C:\Users\hp\Downloads\task 4.exe
Device Name: Samsung Galaxy
Device Function: Multimedia

==== (Samsung Galaxy's Hardware)====
Body Material: Aluminium
Battery Capacity: 4500
Connectivity: Bluetooth
Storage Derive: HDD

====(Samsung Galaxy's Software)====
Operating System: Android
Productivity Software: MS Office
Graphic Software: Adobe Photoshop
Utility Software: CCleaner

Pointer is empty

Device Name: iPhone 17
Device Function: Communication

==== (iPhone 17's Hardware)====
Body Material: Titanium
Battery Capacity: 5000
Connectivity: Wireless
Storage Derive: SSD

====(iPhone 17's Software)====
Operating System: iOS
Productivity Software: Google Docs
Graphic Software: Nvidia Studio
Utility Software: Disk Cleanup
```

Conclusion :

- Classes act like blueprints that define the data and behavior of objects.
- Objects are real instances created from classes, and each object can have its own values while sharing the same structure.
- Composition means one class can hold objects of another class to show a has-a relationship.
- Pointers can be used to manage objects, and a null pointer shows when a pointer is empty
- .Static objects live on the stack and are destroyed automatically, while dynamic objects are created with new and must be deleted manually.
- Always free memory of dynamic objects with delete, while static objects clean up on their own.